CO-2

ASSESSMENT-1

TOOL

Name: Tamil Elakiya S

Reg No: 192524443

Course: Machine Learning

Course code: ITA0612

Slot: A

1. A perceptron model is used to classify whether a loan is approved based on two features: income and credit score. The weights are 0.7 and 0.5 and the bias is −0.6. For an applicant with income = 1.5 and credit score = 2, compute the net input and apply the step activation function. Determine whether the loan is approved or rejected.

Given:

- Income (x_1) = 1.5, weight (w_1) = 0.7
- Credit score (x_2) = 2, weight (w_2) = 0.5
- Bias (b) = −0.6

Sol:

$$\text{Net Input} = w_1x_1 + w_2x_2 + b$$
$$= (0.7 \times 1.5) + (0.5 \times 2) - 0.6$$
$$= 1.05 + 1 - 0.6$$
$$= 1.45$$

Apply step activation function:

$$f(x) = \begin{cases} 1, & \text{if net input} \geq 0 \\ 0, & \text{if net input} < 0 \end{cases}$$

WKT, net input = 1.45

Since, net input ≥ 0, the output is 1.

∴ The loan is Approved.

2. In a neural network, a single neuron uses the delta learning rule. The initial weight is 0.4, learning rate is 0.2, and input is 3. The actual output is 0.5, while the target output is 0.9. Calculate the error, determine whether the loan is approved or not and the weight update, and compute the new weight after one iteration.

Given:

- Initial weight (w) = 0.4
- Learning rate (η) = 0.2
- Input (x) = 3
- Actual output (y) = 0.5
- Target output (t) = 0.9

Sol:

Calculating error:

$$\text{Error} = t - y$$
$$= 0.9 - 0.5$$
$$= 0.4$$

Calculate weight update:

$$\Delta w = \eta (t-y) x$$
$$= 0.2 \times 0.4 \times 3$$
$$\Delta w = 0.24$$

Calculate new weight:

$$w_{new} = w_{old} + \Delta w$$
$$= 0.4 + 0.24$$
$$= 0.64$$

∴ Error = 0.4, weighted update = 0.24, new weight = 0.64.

3. A classification model produces the following confusion matrix values. True Positives = 120, True Negatives = 200, False Positives = 30 and False Negatives = 50. Compute accuracy, precision, recall (sensitivity), specifically and F1-score. Based on these results, evaluate whether the model is suitable for applications where false negatives are critical.

Given:

- True Positives (TP) = 120
- True Negatives (TN) = 200
- False Positives (FP) = 30
- False Negatives (FN) = 50

Sol:

Calculate accuracy:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \times 100$$

$$= \frac{120 + 200}{120 + 200 + 30 + 50} \times 100$$

$$= 80\%$$

Calculate precision:

$$\text{Precision} = \frac{TP}{TP + FP} \times 100$$

$$= \frac{120}{120 + 30} \times 100$$

$$= 80\%$$

Calculate recall (Sensitivity)

$$\text{Recall} = \frac{TP}{TP + FN} \times 100$$

$$= \frac{120}{120 + 50} \times 100$$

$$= 70.59\%$$

Calculate Specificity:

$$\text{Specificity} = \frac{TN}{TN + FP} \times 100$$

$$= \frac{200}{200 + 30} \times 100$$

$$= 86.96\%$$

Calculate F1-score:

$$F1 = \frac{2TP}{2TP + FP + FN} \times 100$$

$$= \frac{240}{240 + 30 + 50} \times 100$$

$$= 75\%.$$

The model has 80% accuracy, but its recall is only 70.59%. It misses 50 actual positive cases. Therefore, it may not be suitable when false negatives are critical.

4. A Genetic Algorithm is used to maximize the function $f(x) = 2x+3$ for an initial population of values $\{1,3,5,7\}$. Compute the fitness of each individual and select the top two candidates. Perform a single-point crossover to generate offspring and explain how mutation can help avoid premature convergence.

Given:

Fitness function: $f(x) = 2x+3$

Initial population: $\{1,3,5,7\}$

Sol:

Calculate fitness:

Individual (x)	fitness $f(x) = 2x+3$
1	$2(1)+3 = 5$
3	$2(3)+3 = 9$
5	$2(5)+3 = 13$
7	$2(7)+3 = 17$

↳ The highest fitness values are 17 and 13.

↳ Selected candidates: 7 and 5.

Parent 1: 7 = 111
Parent 2: 5 = 101

choose a crossover point after the first bit:

- Parent 1 : 1 | 11
- Parent 2 : 1 | 01

so,

- offspring 1 : (1 | 01) = 101 = 5
- offspring 2: (1 | 11) = 111 = 7

Mutation randomly changes one or more bits in an offspring.

∴ The top two candidates are 7 and 5. Crossover produces offspring 5 and 7 in this example, while mutation can introduce new candidate solutions.

5. During training of a neural network, the training loss decreases continuously, while validation loss decreases initially and then starts increasing after several epochs. Identify the issue occuring in the model. Explain the reason behind this behavior and suggest two techniques such as regularization or early stopping to improve generalization performance.

Sol:

Identification: The problem is overfitting.

Explanation:

- Training loss continuously decreases because the model learns the training data.
- Validation loss initially decreases as the model learns useful patterns.
- Later, validation loss increases because the model starts learning noise and specific details of the training data instead of general patterns.

- As a result, the model performs well on training data but poorly on unseen data.

Two techniques to improve generalization:

1.) Regularization:

Add a penalty for large weights, such as L1 or L2 regularization. This helps prevent the model from becoming unnecessarily complex.

2) Early Stopping:

Monitor validation loss during training and stop when it starts increasing. This prevents excessive training and reduces overfitting.

Overfitting can be reduced using regularization and early stopping, improving the model's performance on unseen data.